

AGENTRIX HACKATHON

LangGraph RAG Architecture Escalation Protocol

Mission Brief: Preparing for a high-stakes engineering competition requires moving beyond simple API wrappers. This gauntlet is designed to transition you from linear retrieval scripts to production-grade, stateful multi-agent systems. Master these seven architectures, and you will be equipped to tackle any problem thrown your way.

Phase 1: The Linear Oracle

LEVEL: BEGINNER

CONTEXT

You need to build a straightforward internal documentation bot. A user asks a question about a dense API manual, the bot pulls the most relevant paragraphs from a vector database, and generates a concise answer based strictly on that context.

TECHNOLOGIES

LangGraph (StateGraph), LangChain, Chroma/FAISS, TextSplitters, ChatOpenAI/ChatGoogleGenerativeAI

IMPLEMENTATION STRATEGY

Define a simple TypedDict state containing query, documents, and response. Build exactly two nodes:

- **Retrieve Node:** Takes the query, performs a similarity search on your vector store, and writes the results to documents.
- **Generate Node:** Takes the documents and query, injects them into a system prompt, and calls the LLM.

Connect them sequentially: START → Retrieve → Generate → END.

Phase 2: The Routing Oracle

LEVEL: INTERMEDIATE

CONTEXT

An Intelligent Land Investment decision support system needs to answer user queries. If the user asks about general property zoning laws, it should use the vector database. If the user asks about live interest rates, it needs to bypass the database and use a web search tool.

TECHNOLOGIES

LangGraph (Conditional Edges), Tavily Web Search API, Vector Store, LLM with Tool Binding

IMPLEMENTATION STRATEGY

Introduce dynamic routing before retrieval.

- **Router Node:** An LLM assesses the query. Use structured output to force the LLM to return either "vector_db" or "web_search".
- **Conditional Edge:** Based on the router's output, route the flow to either the Vector_Retrieve node or the Web_Search node.
- **Convergence:** Both retrieval nodes flow into a single Generate node to synthesize the final answer.

Phase 3: Corrective RAG (CRAG)

LEVEL: ADVANCED INTERMEDIATE

CONTEXT

You are building a financial compliance bot. Vector searches frequently pull up outdated or irrelevant tax codes. If the retrieved documents are poor quality, the system must discard them, rewrite the user's query for better SEO, and perform a live web search before answering.

TECHNOLOGIES

LangGraph, LangChain Prompts, DuckDuckGo Search API, LLM Evaluators

IMPLEMENTATION STRATEGY

Implement a grading loop.

- **Retrieve Node:** Fetch docs from the vector DB.
- **Grade Node:** A lightweight LLM checks if the docs actually answer the query. Updates state with a `relevant: yes/no` flag.
- **Conditional Edge:** If yes, go to Generate. If no, route to a `Rewrite_Query` node.
- **Fallback Flow:** The `Rewrite_Query` node optimizes the string and passes it to a `Web_Search` node, which then flows to Generate.

Phase 4: Self-Reflective RAG (Self-RAG)

LEVEL: ADVANCED

CONTEXT

Hallucinations are unacceptable in your hackathon pitch. You need an agent that reviews its own generated answers. If the answer contains facts not present in the retrieved documents, it must throw the answer away and try again.

TECHNOLOGIES

LangGraph (Cyclic Graphs), LangChain Output Parsers, Strict System Prompts

IMPLEMENTATION STRATEGY

Add a post-generation cyclic validation loop.

- After the Generate node, route to a Hallucination_Grader node.
- The grader compares the response strictly against the documents.
- **Conditional Edge:** If `hallucination == true`, route backward to the Generate node (or even the Rewrite node) to try again. Provide the LLM with the previous failure reason.
- If `hallucination == false`, route to END.

Phase 5: The Microservices Multi-Agent

LEVEL: EXPERT

CONTEXT

You need a backend system that interfaces with a Next.js frontend dashboard. The user requests a comprehensive competitive analysis. You need one "Researcher" agent to scrape the web, one "Data Analyst" agent to query your SQL database, and a "Supervisor" to delegate tasks and compile the final report.

TECHNOLOGIES

LangGraph (Subgraphs / Multi-Agent Supervisor Pattern), SQLDatabaseChain, Web Scrapers

IMPLEMENTATION STRATEGY

Treat each agent as a node or a compiled Subgraph.

- **Supervisor Node:** Uses an LLM to determine who needs to work next. It outputs the name of the next worker (e.g., "Researcher", "Analyst", or "FINISH").
- **Worker Nodes:** Execute their specific tools, update the shared state's messages list, and route back to the Supervisor.
- This creates a hub-and-spoke graph architecture where the Supervisor orchestrates the workers until the task is complete.

Phase 6: Stateful Memory & Profile Building

LEVEL: EXPERT

CONTEXT

The judges want a personalized experience. Build a conversational tutor that remembers what the user struggled with 30 minutes ago. As the user chats, the bot retrieves subject matter from a vector DB, but also silently updates a JSON "User Profile" stored in a persistent database.

TECHNOLOGIES

LangGraph (MemorySaver / Checkpointing), Redis/SQLite, JSON Parsers

IMPLEMENTATION STRATEGY

Introduce Thread Checkpointing.

- Define your state with messages (using the `add_messages` reducer) and a `user_profile` dictionary.
- Compile your graph with a checkpointer (e.g., `MemorySaver()` or a SQLite checkpointer).
- **Profile Updater Node:** Runs in parallel or right after the generation node. It reads the chat history and updates the `user_profile` key with inferred preferences or skill levels, ensuring continuity across API calls.

Phase 7: Human-in-the-Loop (HITL) Action Protocol

LEVEL: MASTER

CONTEXT

For TheFuriousFive's final hackathon submission, you are building an automated HR recruiting system. The RAG agent retrieves candidate resumes, scores them against job requirements, and drafts an acceptance or rejection email. However, the system is forbidden from actually sending the email without human approval.

TECHNOLOGIES

LangGraph (Interrupts / breakpoints), Email API (SendGrid/SMTP)

IMPLEMENTATION STRATEGY

Utilize graph breakpoints.

- Compile the graph with `interrupt_before=["Send_Email_Node"]`.
- The graph executes the retrieval, scoring, and drafting nodes, then halts.
- The frontend reads the state, displays the drafted email to the human user, and provides "Approve" or "Reject/Edit" buttons.
- The user submits their feedback, and you resume the graph execution by passing the updated state (e.g., the human's edits) back into the `invoke()` method with the same thread ID.